

Tag 2

Benutzereingaben, Umwandlung & erste Fehler

Python-Grundlagen-Schulung

Was du heute lernst

1. Kurze Wiederholung von Tag 1
2. Wie dein Programm den **Benutzer etwas fragen** kann
3. Drei Fehler, die **absichtlich** provoziert werden – zum Verstehen
4. Wie du Text in **Zahlen umwandelst**
5. Viel Übung in vier verschiedenen Kategorien

Teil 1

Kurze Wiederholung

Drei Aufgaben zum Aufwärmen

Erinnerst du dich noch an Tag 1?

- Variablen mit passendem Datentyp anlegen
- Mit festen Zahlen rechnen (Netto, Steuer, Brutto)
- Werte mit einem f-String in einen Satz einbauen

Alle drei Aufgaben findest du im **Colab-Notebook**.

Ein Beispiel zur Erinnerung

```
mitarbeiter_name = "Julia Schmidt"  
personalnummer = 4471  
rolle = "Systemadministratorin"  
  
print(f"{mitarbeiter_name} (Personalnummer {personalnummer}) "  
      f"arbeitet als {rolle}.")
```

Ein f-String verbindet Text und Variablen
in einem einzigen, lesbaren Satz.

Teil 2

Der Benutzer redet mit!

`input()`

Bisher: alles fest im Code

Bisher standen alle Werte schon **fest** in deinem Code.

Ab heute kann dein Programm den **Benutzer selbst fragen** –
und mit dessen Antwort weiterarbeiten!

So funktioniert input()

```
benutzer = input("Bitte geben Sie Ihren Benutzernamen ein: ")  
print(f"Eingabe registriert für Benutzer: {benutzer}")
```

1. Python zeigt den Text in den Klammern an
2. Das Programm **wartet**, bis der Benutzer etwas eintippt
3. Nach **Enter** wird die Eingabe in der Variable gespeichert

Fehler-Sollbruchstelle 1

Der Additions-Absturz

Wir provozieren jetzt absichtlich einen Fehler!

Das machen wir ganz bewusst –
um zu verstehen, was `input()` **wirklich** liefert.

```
alter = input("Wie alt bist du? ")  
alter_in_zehn_jahren = alter + 10  
print(alter_in_zehn_jahren)
```

Gib bei der Abfrage z. B. `25` ein und schau, was passiert.

Die Fehlermeldung

```
TypeError: can only concatenate str (not "int") to str
```

str = String = Text

int = Integer = Ganzzahl

concatenate = verknüpfen / aneinanderhängen

Was ist wirklich passiert?

`input()` liefert **immer** einen Text zurück –

auch wenn der Benutzer `25` eintippt,
speichert Python das als **Text** `"25"`, nicht als Zahl!

Für Python lautet die Rechnung dann: `"25" + 10`

Text und Zahl kann man nicht einfach so addieren.

Merksatz

**input() liefert IMMER Text zurück –
selbst wenn eine Zahl eingetippt wurde.**

Teil 3

Typumwandlung: Text zu Zahl machen

Die Reparatur: int() und float()

`int(...)` wandelt Text in eine **Ganzzahl** um

`float(...)` wandelt Text in eine **Dezimalzahl** um

```
alter = int(input("Wie alt bist du? "))  
alter_in_zehn_jahren = alter + 10  
print(f"In 10 Jahren bist du {alter_in_zehn_jahren} Jahre alt.")
```

Von innen nach außen lesen

```
alter = int( input("Wie alt bist du? ") )
```

 ↑ 1. zuerst
 ↑ 2. danach

Python arbeitet **von innen nach außen**:

Erst wird `input()` ausgeführt (Text kommt rein),
dann wandelt `int()` diesen Text in eine Zahl um.

Fehler-Sollbruchstelle 2

Die ungültige Zahl

Was, wenn keine Zahl eingegeben wird?

```
alter = int(input("Wie alt bist du? "))  
alter_in_zehn_jahren = alter + 10  
print(f"In 10 Jahren bist du {alter_in_zehn_jahren} Jahre alt.")
```

Gib bei der Abfrage das Wort "**dreißig**" ein (in Buchstaben).

Was, glaubst du, passiert jetzt?

Die Fehlermeldung

```
ValueError: invalid literal for int() with base 10: 'dreißig'
```

ValueError = Wert-Fehler

`int()` versucht, aus Buchstaben eine Zahl zu machen –
aber `"dreißig"` enthält gar keine Ziffern!

`int()` funktioniert nur, wenn der Text
wie eine Zahl aussieht (z. B. `"30"`).

Fehler-Sollbruchstelle 3

Die vergessene Klammer

Zwei Klammern auf, zwei Klammern zu

```
port = int(input("Geben Sie den Port ein (z. B. 80):")  
print(f"Port {port} wird geöffnet.")
```

Siehst du den Fehler? Es fehlt eine **schließende Klammer**!

Die Fehlermeldung

```
SyntaxError: unexpected EOF while parsing
```

(manchmal auch: `SyntaxError: '(' was never closed`)

EOF = "End of File" – das Ende der Datei

Python hat das ganze Dokument durchsucht
und **immer noch** auf die fehlende Klammer gewartet!

So wird es repariert

```
port = int(input("Geben Sie den Port ein (z. B. 80): "))  
print(f"Port {port} wird geöffnet.")
```

✓ Beide öffnenden Klammern haben jetzt ihre passende schließende Klammer.

Tipp: Zähle beim Schachteln von Funktionen
(und) immer gegen!



Kurze Pause

Teil 4

Jetzt du: der große Übungs-Pool

Vier Kategorien

A – Reine Text-Interaktion

Kein Umwandeln nötig: Begrüßung, E-Mail-Generator

B – Ganzzahl-Berechnungen

Mit `int()` : Verdoppler, Server-Rack-Planer

C – Nachkommastellen-Berechnungen

Mit `float()` : Mehrwertsteuer, Temperatur-Konverter

D – IT-Support-Herausforderungen

Subnetz-Rechner, Budget-Aufteilung

Ein Beispiel aus Kategorie A

```
vorname = input("Vorname: ")  
stadt = input("Stadt: ")  
print(f"Hallo {vorname}, wie ist das Wetter in {stadt}?")
```

Hier brauchst du **kein** `int()` oder `float()` –
Text bleibt einfach Text.

Ein Beispiel aus Kategorie B

```
belegt = int(input("Wie viele HE sind bereits belegt? "))  
frei = 42 - belegt  
print(f"Es sind noch {frei} Höheneinheiten im Rack frei.")
```

Sobald du **rechnen** willst, brauchst du `int()` oder `float()` .

Ein Beispiel aus Kategorie C

```
netto = float(input("Netto-Preis in Euro: "))  
brutto = netto * 1.19  
print(f"Der Brutto-Preis beträgt {brutto} Euro.")
```

Bei Dezimalzahlen (z. B. 45.90) brauchst du `float()`
statt `int()` .

Ein Vorgeschmack auf Kategorie D

```
budget = int(input("Gesamtbudget der IT-Abteilung: "))  
projekte = int(input("Anzahl der Projekte: "))  
  
betrag_pro_projekt = budget // projekte  
restbetrag = budget % projekte
```

// und % kennst du schon von Tag 1 –
heute kombinierst du sie mit input() .

Zum Schluss

Typische Stolperfallen heute

Woran du oft scheiterst – und das ist okay!

- ✗ Komma statt Punkt bei Dezimalzahlen eingeben
(Python will `45.90` , nicht `45,90`)
- ✗ Vergessen, `input()` in `int()` oder `float()` einzupacken, bevor gerechnet wird
- ✗ Beim Schachteln von Klammern eine vergessen

Was du heute gelernt hast

- ✓ `input()` lässt den Benutzer etwas eingeben
- ✓ `input()` liefert **immer Text** zurück
- ✓ `int()` und `float()` wandeln Text in Zahlen um
- ✓ Fehlermeldungen wie `TypeError`, `ValueError`, `SyntaxError` sind Hinweise, keine Katastrophen

Morgen: Fehler finden & Texte zerschneiden 🎉